

PULL REQUEST · #47 · FEATURE/NEO4J-GRAPHDB-SKELETON

Neo4j as a selectable graph DB engine via Bolt

End-to-end demo on fevm-mjolnir with a real PFAS research-paper ontology — 303 triples written into Neo4j over the Bolt protocol, 99 inferred via OWL 2 RL, paper itself never leaves the workspace.



v0.6 Neo4j slot — landed, end-to-end

OntoBricks now supports **Neo4j (Bolt / Cypher)** as a selectable Graph DB engine alongside Lakebase Postgres. Opt-in via Settings → Triple Store → Global → Neo4j (Bolt). Lakebase remains the default; no breaking changes.

32

classes generated from your PFAS paper by the AI ontology generator

303

RDF triples materialized into Neo4j by the OntoBricks build pipeline

99

inferred triples from T-Box OWL 2 RL reasoning, in 0.102 s

10.3 s

full build time: prepare → view → graph

- **C1 / C2 preserved** — Writes only via the OntoBricks build pipeline. `Neo4jStore.execute_query` raises `NotImplementedError` by design — no raw Cypher entry point.
- **Cleanup done** — `WaterTreatment_V1` label dropped from the demo Neo4j after capture. Total nodes: **0**. Paper stays in `fevm-mjolnir`.

Why now

3 / 21

Benoit's v0.5 introduced the GraphDBBackend abstraction. The v0.6 roadmap is to land Neo4j as the first non-Lakebase backend, originally targeted at August 2026. This PR delivers it ahead of schedule with a complete user-visible path: pick the engine in Settings, build, query.

C1 OntoBricks controls the graph

Single canonical source of truth = the OntoBricks build pipeline. Mirror modes, dual-writes, etc., were explicitly ruled out in the 2026-05-20 sync.

C2 The ontology controls writes

No raw Cypher entry point exposed to users.
`execute_query` raises `NotImplementedError`
— all writes go through validated R2RML.

C3 No raw data in the graph

Only ontology-shaped triples land in Neo4j. The synthetic UC fact table stays in Unity Catalog; Neo4j receives the resolved entities.

- **Customer value** — Customers already on Neo4j (Ryan's contacts, EU manufacturing) get an OntoBricks deployment without forcing Lakebase.
- **Architectural credibility** — Proves the GraphDBBackend abstraction is real and ships with two backends, not one and a placeholder.

Factory dispatch, two backends

4 / 21

Engine selection is resolved once via `TripleStoreFactory._resolve_graph_engine` against `GlobalConfigService`. The factory dispatches to either `LakebaseStore` (existing) or the new `Neo4jStore`. Same `GraphDBBackend` interface, different SQL/Cypher dialect.



- Single switch in Settings, no per-domain override on this PR.
- Capability flags on `Neo4jStore`: `supports_cypher=True`, `query_dialect="cypher"`, `supports_graph_model=False` (flat triple in v1).
- Reasoning translator is symmetric: `SWRLSQLTranslator` ↔ `SWRLFlatCypherTranslator` (scaffolded — full Cypher in a follow-up PR).

What's in the PR — backend

FILE	PURPOSE
<code>back/core/graphdb/neo4j/Neo4jStore.py</code> NEW · ~580 LOC	Full <code>GraphDBBackend</code> impl. 16 named-query methods in native Cypher. <code>execute_query</code> raises <code>NotImplementedError</code> (C2).
<code>back/core/graphdb/GraphDBFactory.py</code>	<code>_create_neo4j</code> dispatch + <code>NEO4J_AVAILABLE</code> guarded import.
<code>back/objects/session/GlobalConfigService.py</code>	<code>ALLOWED_GRAPH_ENGINES</code> = <code>("lakebase", "neo4j")</code> .
<code>back/core/reasoning/SWRLFlatCypherTranslator.py</code> NEW	Scaffolded mirror of SQL translator. Returns <code>None</code> + logs warning. Reasoning reports 0 violations / 0 inferences gracefully until the full translator lands.
<code>pyproject.toml</code>	<code>[project.optional-dependencies]</code> <code>neo4j</code> = <code>["neo4j ≥ 5.0"]</code> .
<code>app.yaml.template</code>	Adds <code>--extra neo4j</code> so the deployed App ships the driver.

What's in the PR — frontend

6 / 21

FILE	PURPOSE
front/templates/settings.html	<code><option value="neo4j">Neo4j (Bolt)</option></code> + Neo4j config form (URI, db, auth, creds, encrypted).
front/config/menu_config.json	"Neo4j" item in the Settings sidebar.
front/static/config/js/settings.js	Save/load wiring for the Neo4j engine config.
front/static/domain/js/domain-validation.js	Pre-Build, async-fetch <code>/settings/graph-engine</code> so Validation card title is engine-aware.
front/static/query/js/query-sync.js	Same engine-aware fallback for the Build page title.
front/fastapi/dependencies.py ROOT CAUSE FIX	<code>triplestore_page_context</code> previously hard-coerced every engine to <code>"lakebase"</code> (<code>_raw if _raw == "lakebase"</code> else <code>"lakebase"</code> — a tautology). Now passes the resolved engine through directly.

The server-side fix + the two JS fetch fallbacks are defence in depth: page works correctly even on a stale `__TRIPLESTORE_CONFIG.graph_engine`.

The path — paper to triples

7 / 21

Real research paper goes in, real RDF triples come out the other side of the OntoBricks build pipeline, written into Neo4j over Bolt via the new Neo4jStore. Synthetic 12-row UC fact table provides the ABox instances — the only fake bit; numbers are illustrative.

1 Settings

Switch global engine to Neo4j (Bolt). Fill Bolt URI + creds + encrypted on.

2 Documents + Generate

Upload `AV-TR-2026-001_MMSF-PFAS-Risk-Assessment.pdf`. AI generates a 32-class ontology.

3 Data Source

Create + load

`mjolnir_catalog.ontobricks.pfas_measurements_demo`
(12 rows, paper-aligned schema).

4 Auto-Map

Batch run maps 25/25 entities and 12/12 relationships. 7 abstract entities excluded (no anchor column).

5 Diagnostics + Build

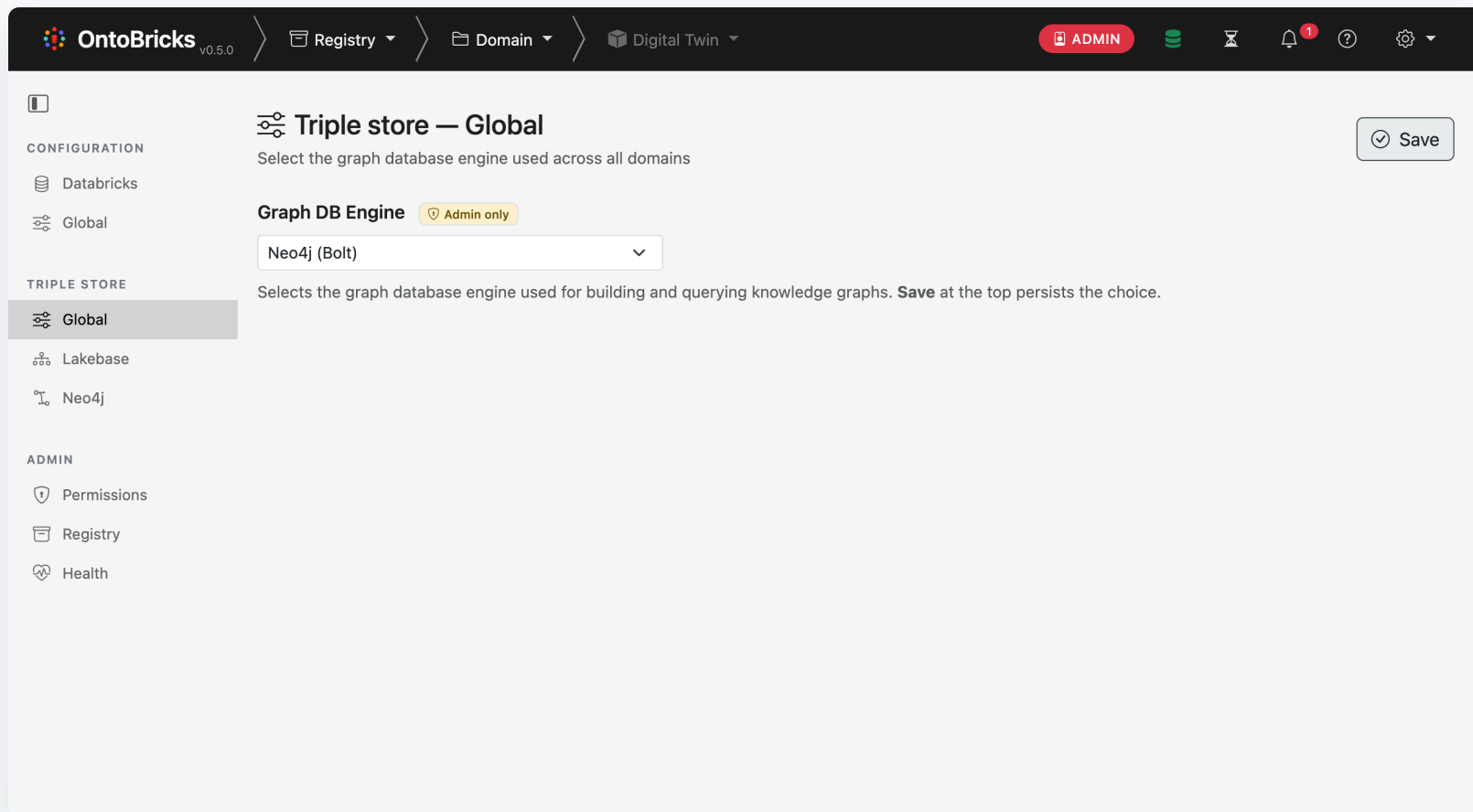
0 errors. Build pipeline emits 303 triples in 10.3 s into the new Neo4j backend.

6 Verify + Inference

Neo4j Browser shows 303 nodes. T-Box reasoning produces 99 inferred triples in 0.1 s.

Switch engine to Neo4j

8 / 21



Triple Store → Global → **Neo4j (Bolt)** · saved confirmation toast. 01-settings-global-neo4j-saved.png

Neo4j connection form

9 / 21

The screenshot shows the OntoBricks v0.5.0 web interface. The top navigation bar includes the OntoBricks logo, version, and breadcrumbs: Registry > Domain > Digital Twin. On the right of the bar are links for ADMIN, a home icon, a clock, a notification bell with a red '1', a help icon, and a settings gear. The left sidebar contains a 'CONFIGURATION' section with 'Databricks' and 'Global', a 'TRIPLE STORE' section with 'Global' and 'Lakebase', and an 'ADMIN' section with 'Permissions', 'Registry', and 'Health'. The 'Neo4j' configuration page is active, showing a title 'Neo4j' and a subtitle 'Configure the Neo4j (Bolt) connection — Aura or self-hosted'. At the top right of the form are 'Test connection' and 'Save' buttons. A light blue informational box contains an 'Architecture note' about writing triples via Bolt and ontology validation. The form fields include: 'Bolt URI' with a red asterisk and a value 'neo4j+s://b4810af7.databases.neo4j.io'; a note about Aura and self-hosted URI schemes; 'Database' with a value 'neo4j' and a default note; 'Auth method' with a dropdown set to 'Basic (username + password)'; 'Username' with a value 'neo4j'; and 'Password' with a masked field. A checkbox for 'Encrypted connection' is checked, with a note about URI schemes. A final light blue box at the bottom states: 'Test-connection is not wired up yet. Save the config and run a build to verify the connection.'

Bolt URI · basic auth · encrypted on. Form is engine-generic — works against any Bolt endpoint (Aura, AuraDS, self-hosted, on-prem). Test-connection is a stub for now — "Save & run a build to verify".

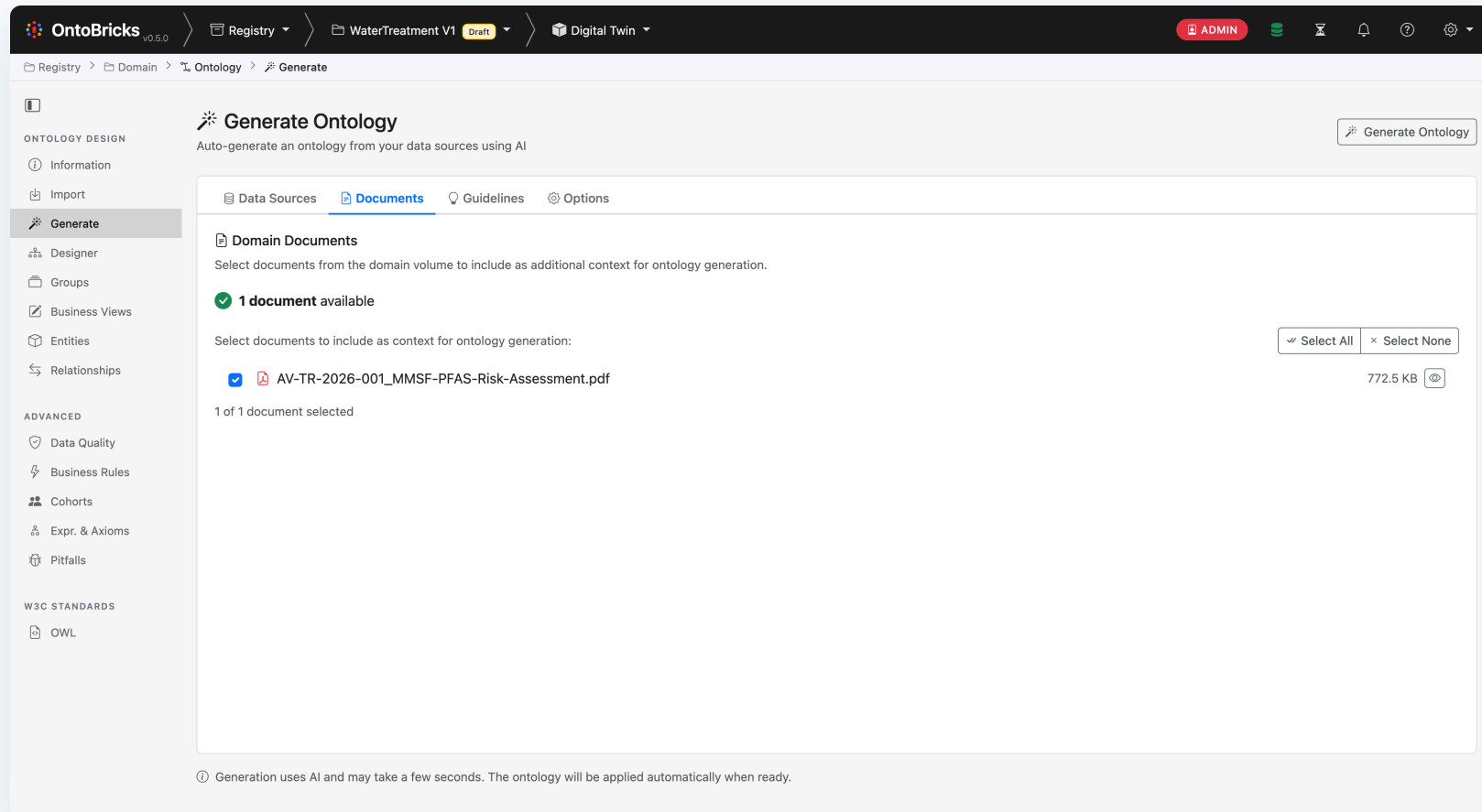
■ OntoBricks v0.6 · PR #47

02-settings-neo4j-form-filled.png

2026-06-12

PDF in, ontology out

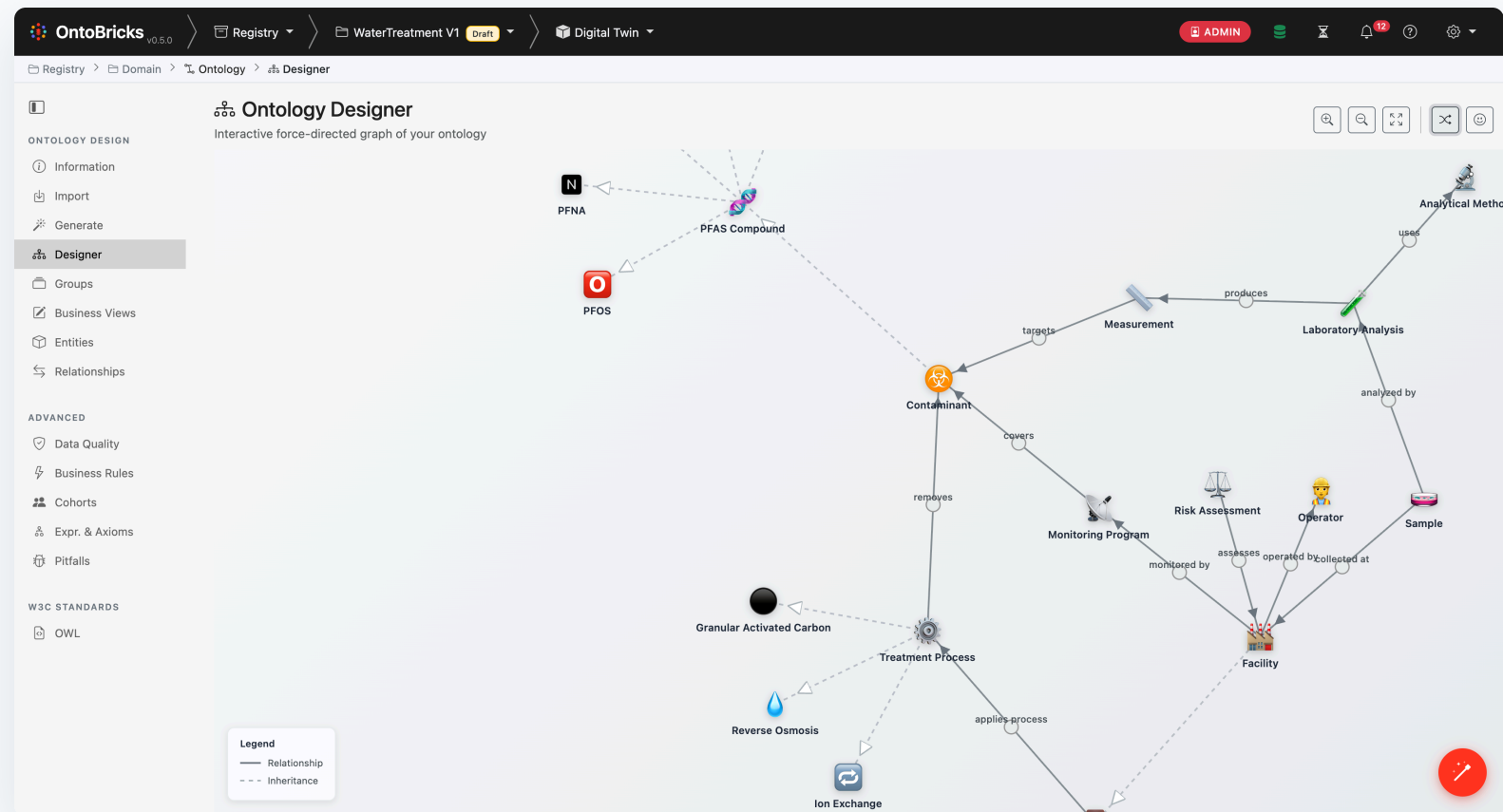
10 / 21



Generate Ontology → Documents tab · PFAS Risk Assessment PDF selected as AI context. 03-ontology-generate-pdf-selected.png

32 classes, 13 relations

11 / 21



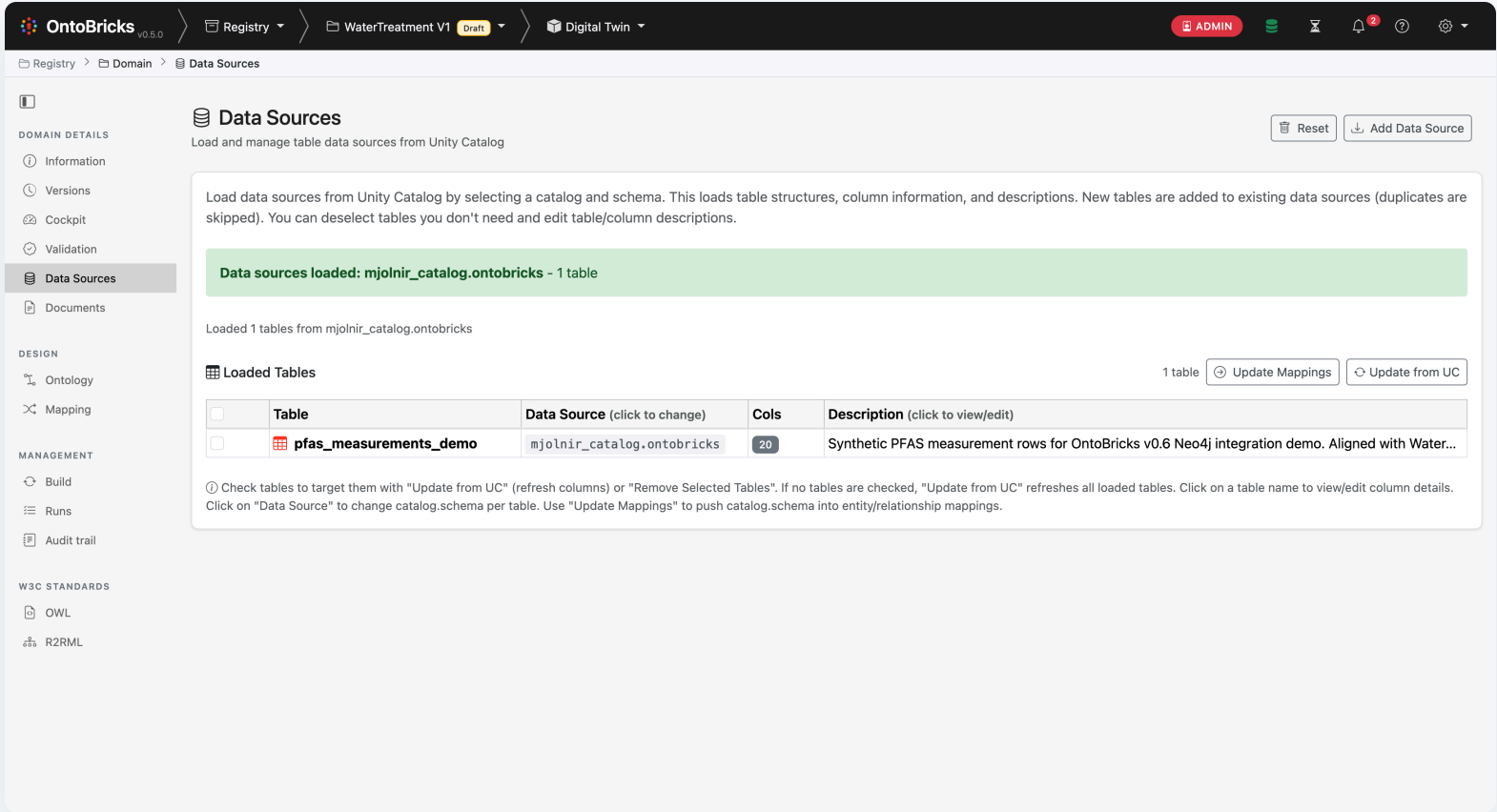
PFAS Compound + PFOA/PFOS/PFNA/PFHxS/GenX subclasses · Treatment Facility · Treatment Process + GAC/IX/RO · Water Source + Groundwater/Surface · Risk Assessment · Measurement · Regulation · Operator · Monitoring Program · etc.

■ **OntoBricks v0.6 · PR #47**

04-ontology-designer-with-icons.png

2026-06-12

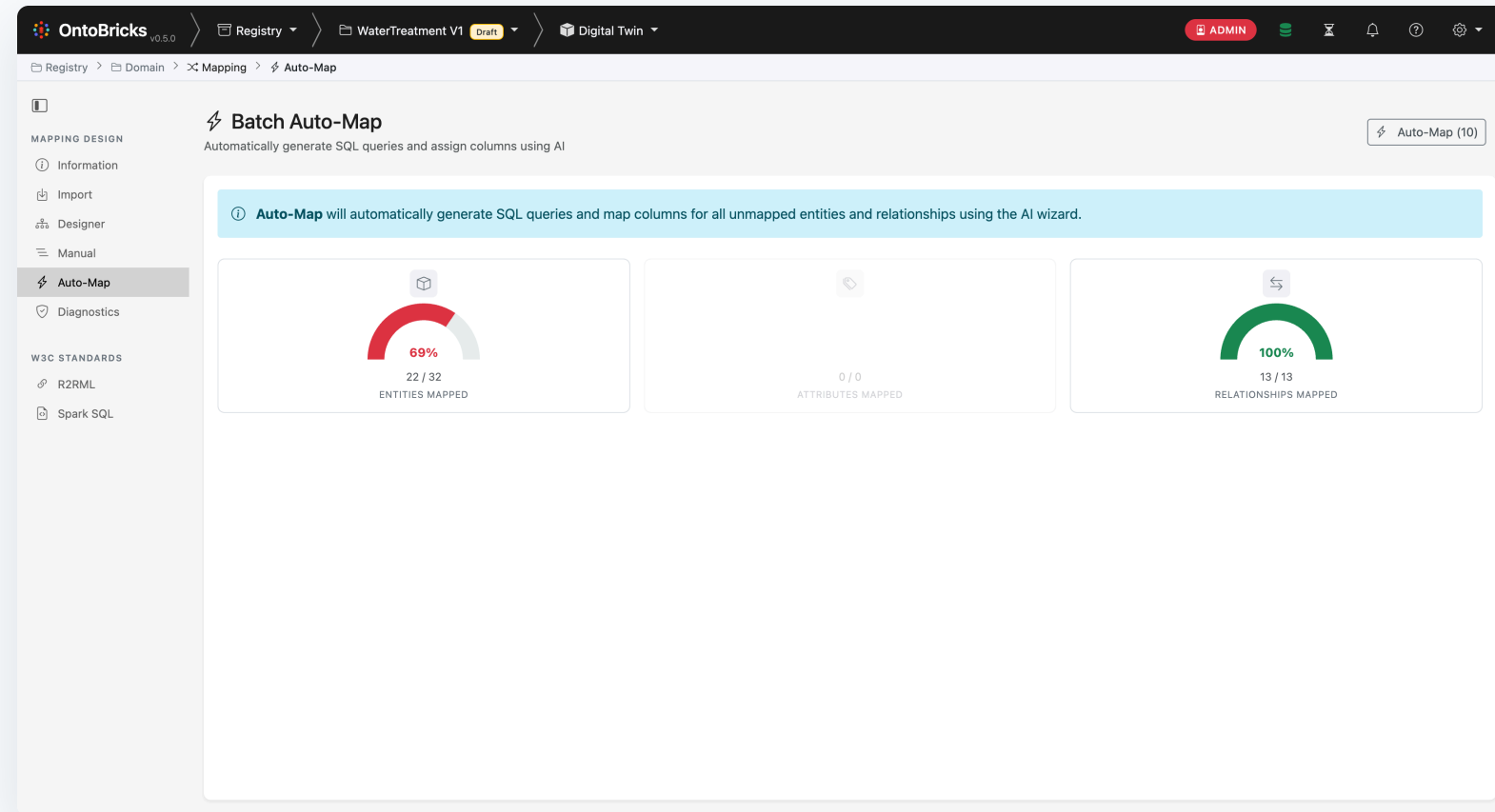
Synthetic UC fact table



mjoinir_catalog.ontobricks.pfas_measurements_demo · 12 rows · 20 columns · synthetic but paper-aligned schema (facility / water source / process / contaminant / measurement / regulation).

25 entities + 13 relations mapped

13 / 21



AI batch Auto-Map · 22 entities + 13 relations on first pass · 3 more added via single-entity Auto-Map (Sample, Treatmentfacility, Riskassessment) · 7 abstract classes excluded as designed.

OntoBricks v0.6 · PR #47

07-automap-completed-
22of32.png

2026-06-12

303 triples in Neo4j · 5.3 s

14 / 21

OntoBricks v0.5.0 Registry > Domain > Build

Digital Twin Information
Triple store status and build execution

Warning: Ontology or assignments have changed since the last Digital Twin build. The triple store may be out of date. Click **Build** to regenerate it with the latest configuration.

Readiness Status
Mapping: Ready

Digital Twin
 Last Built: 12 Jun 2026, 12:27:13
 Last Updated: 12 Jun 2026, 14:08:19
 Graph contains 303 triples

3-card architecture flow:

- Triple Store** (Exists) → **Bolt** (Cypher UNWIND · MERGE) → **Graph DB (Neo4j)** (Exists)
- Triple Store path: `mjoinir_catalog.ontobricks.triplestore_watertreatment_v1`
- Bolt action: `Cypher write at build time`
- Graph DB path: `WaterTreatment_V1`

3-card architecture: Triple Store → Bolt (Cypher UNWIND · MERGE) → Graph DB (Neo4j) · WaterTreatment_V1. The Bolt card is the Neo4j-specific bridge (mirrors the Lakeflow Sync card on Lakebase). Build wrote 303 triples.

OntoBricks v0.6 · PR #47

10-build-success-303-triples-neo4j.png

2026-06-12

Neo4j Browser — the triples

15 / 21

The screenshot displays the Neo4j Browser interface. On the left, the 'Database information' sidebar shows 303 nodes, with 'WaterTreatment_V1' highlighted. Below this, a list of property keys is visible, including 'amount_eur', 'claim_id', 'created_at_utc', 'created_by', 'date_of_birth', 'email_hash', 'inferred', 'make', 'model', 'name', 'note', 'object', 'opened_at', 'policy_number', 'predicate', 'predicate_uri', 'premium_eur', 'rule', 'segment', and 'session_marker'. The main area shows a Cypher query: `neo4j$ MATCH (n:WaterTreatment_V1) WHERE n.predicate ENDS WITH "#type" RETURN n LIMIT 25`. The results are displayed as a graph with 25 red nodes. On the right, the 'Results overview' panel shows 'Nodes (25)' and 'WaterTreatment_V1 (25)'. The bottom of the interface features a navigation bar with 'GUIDE', 'DATASET', and 'Share your feedback' links.

Direct Cypher query against the live Neo4j · **Nodes (303)**, all under single label `:WaterTreatment_V1` · 25 `rdf:type` nodes rendered (one per ontology class instantiated).

■ **OntoBricks v0.6 · PR #47**

12-neo4j-browser-303-nodes-graph.png

2026-06-12

Inference — 99 triples in 0.1 s

16 / 21

OntoBricks v0.5.0 > Registry > WaterTreatment V1 Draft > Digital Twin > Inference

Inference
OWL 2 RL inference, SWRL rules, graph reasoning, and business rules
Domain: WaterTreatment v1 Draft [Switch domain](#) [Run Inference](#)

Options [Report](#) Inferred 99

Duration: 0.102s | Inferred: 99

Phase	Status	Duration	Details
T-Box (OWL 2 RL)	OK	0.102s	99 inferred
SWRL Inference	Skipped	—	No SWRL rules defined in ontology
Graph Inference	OK	—	No new findings
Decision Tables	Skipped	—	No decision tables defined
SPARQL CONSTRUCT Rules	Skipped	—	No SPARQL rules defined
Aggregate Rules	Skipped	—	No aggregate rules defined
Total	OK	0.102s	99 inferred (+ 99 schema/structural)

Executed at 12/06/2026, 10:31:41

Materialise inferred triples

☐ Write to Delta table

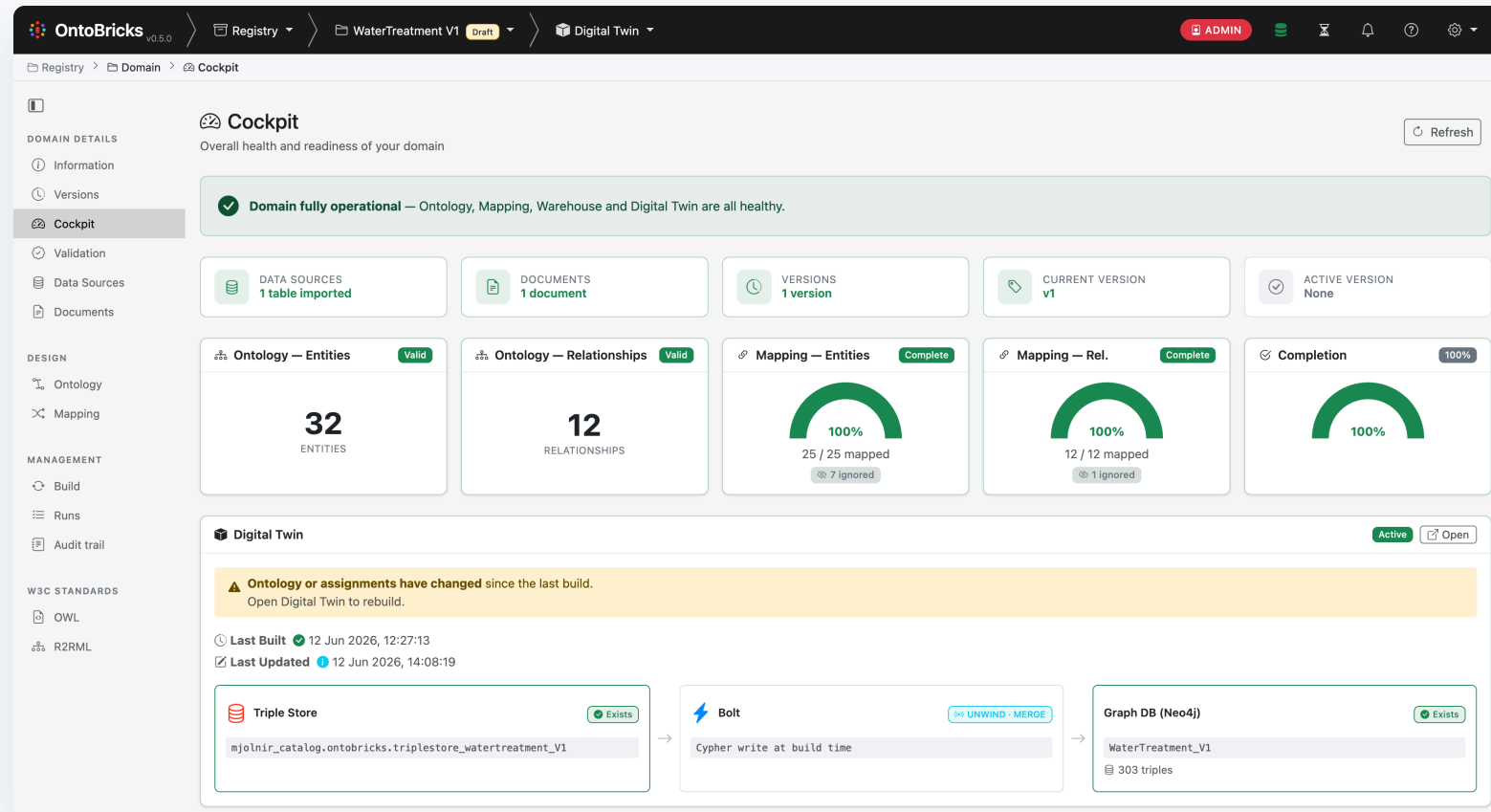
☒ Append to graph database (Graph DB)

[Materialise](#)

T-Box (OWL 2 RL) ✓ 99 inferred · SWRL skipped (scaffold no-op as designed) · others skipped gracefully (no rules in this domain). 15-inference-99-inferred.png

Cockpit — fully operational on Neo4j

17 / 21



Cockpit now visibly shows the active engine via the 3-card arch: Triple Store → Bolt (UNWIND · MERGE) → Graph DB (Neo4j) · 303 triples. Before this PR the Cockpit was entirely engine-agnostic.

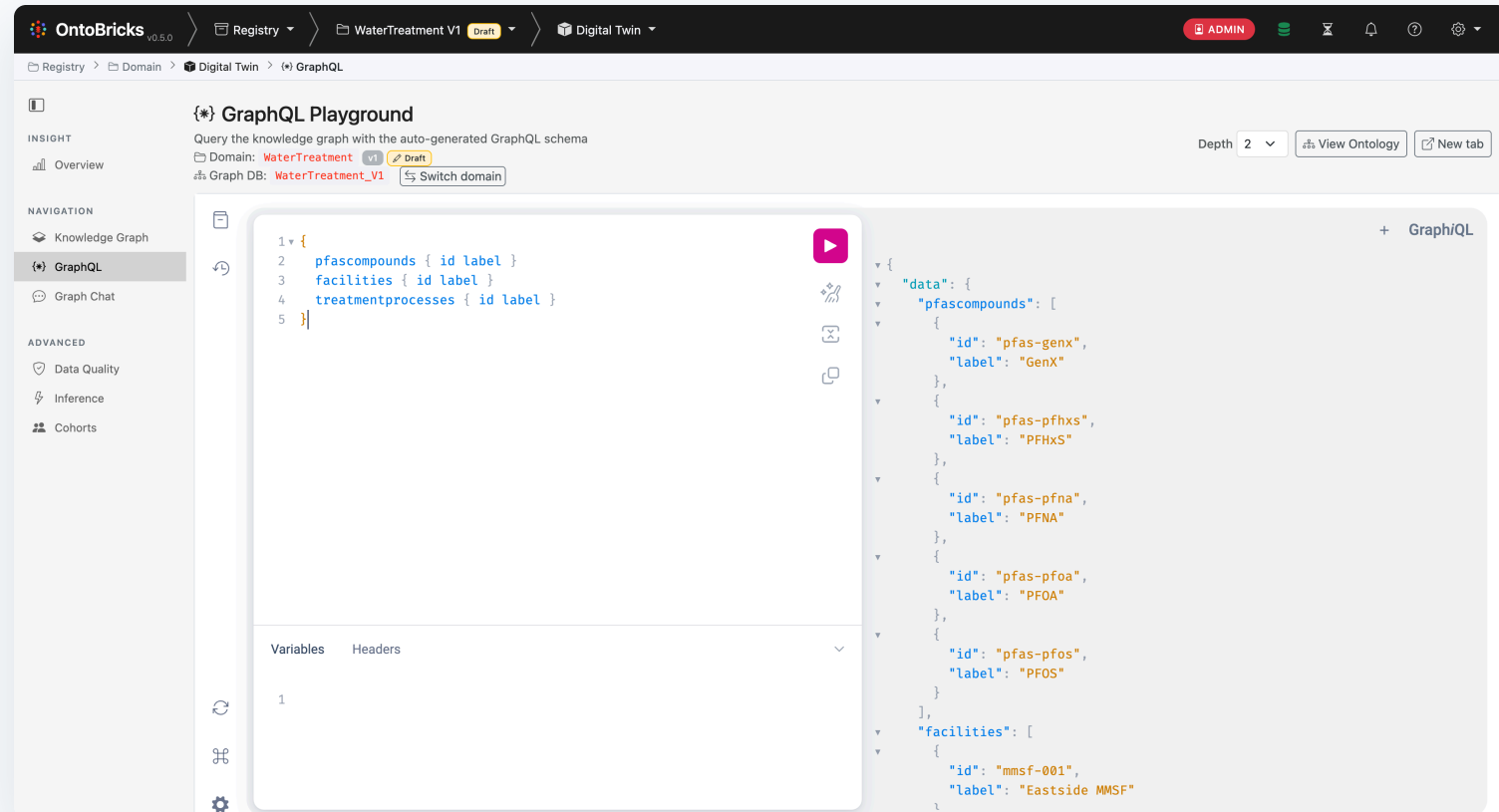
■ **OntoBricks v0.6 · PR #47**

16-cockpit-neo4j-active.png

2026-06-12

GraphQL Playground over the Neo4j graph

18 / 21



Query { pfascompounds { id label } facilities { id label } treatmentprocesses { id label } } resolved transparently: **5 PFAS compounds** (GenX, PFHxS, PFNA, PFOA, PFOS), **3 facilities** (Eastside / Westport / Northvale MMSF), **6 treatment processes**. The GraphQL service rewrites this into Cypher under the hood and queries Neo4j.

■ **OntoBricks v0.6 · PR #47**

17-graphql-playground-watertreatment.png

2026-06-12

GraphQL → Cypher · how the resolver talks to Neo4j

19 / 21

GraphQL playground sends `{ pfasc compounds { id label } }`. The OntoBricks resolver calls two named-query methods on `Neo4jStore`. **Every value is bound** — no string interpolation.

1 `Neo4jStore.find_subjects_by_type(...)`

```
MATCH (t:`WaterTreatment_V1`)
WHERE t.predicate = $rdf_type
      AND t.object   = $type_uri
RETURN DISTINCT t.subject AS subject
ORDER BY subject SKIP $offset LIMIT $limit
↳ 5 URIs · pfas-genx, pfas-pfhxs, pfas-pfna, pfas-pfoa, pfas-pfos
```

2 `Neo4jStore.get_entity_metadata(...)`

```
MATCH (t:`WaterTreatment_V1`)
WHERE t.predicate = $rdfs_label
      AND t.subject IN $subjects
RETURN t.subject AS subject, t.object AS label
↳ 5 labels · GenX, PFHxS, PFNA, PFOA, PFOS
```

WHY THIS MATTERS

C2 enforced in code, not just in the doc

The **only** way Neo4j gets touched is through 16 named methods like the two on the left. `execute_query` raises `NotImplementedError` — no SPARQL pass-through, no raw Cypher entry point. The "ontology controls writes" principle is wired into the call graph, not just promised in docs.

Zero injection surface

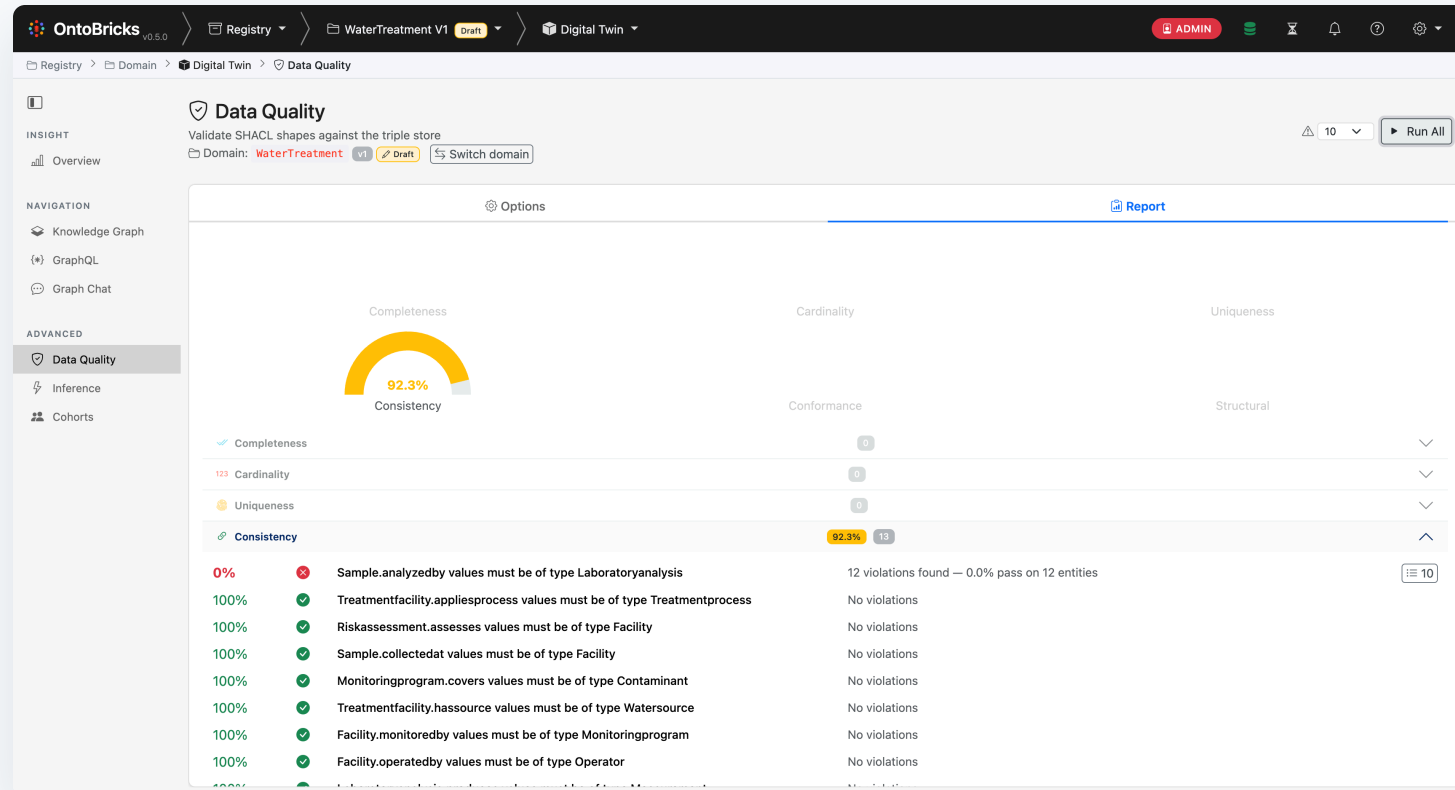
Every value (`$rdf_type` , `$type_uri` , `$subjects`) is bound via the Neo4j driver's parameter map. No `f"..."` formatting, no `str.format` , no `+` . User input never reaches the Cypher parser as text.

Flat triple model — by design, explains slide 15

Each triple = **one node** with `subject` / `predicate` / `object` properties. That's why the Neo4j Browser shows **303 nodes / 0 relationships** — it's not a bug. Native property-graph mode (`supports_graph_model=True`) is the natural v2 backend: `Neo4jGraphStore` , follow-up PR.

SHACL Data Quality — Graph mode on Neo4j

20 / 21



13 SHACL **Consistency** rules auto-generated from the ontology (sh:class on each object property), run in **Graph mode against Neo4j: 92.3% pass** — 12/13 rules at 100%, one rule (Sample.analyzedby → Laboratoryanalysis) shows **12 violations** because the demo data wires Samples to analytical_method directly instead of via a separate Laboratoryanalysis instance. Exactly the kind of signal SHACL validation is built for.

■ **OntoBricks v0.6 · PR #47**

18-data-
quality-
graph-on-
neo4j.png
2026-06-12

Bugs found · limitations · ask

21 / 21

Bugs found & fixed in this PR

- **Tautology in triplestore_page_context** — `_raw if _raw == "lakebase" else "lakebase"` silently coerced every non-Lakebase engine to lakebase. Now passes through.
- **Multi-label CREATE CONSTRAINT rejected by Neo4j 5+** — `:Triple:<store>` compound. Switched to single backtick-quoted label.
- **Driver missing in deployed App** — `app.yaml.template` `uv-run` lacked `--extra neo4j`. Added.

Known limitations (deliberate)

- **SWRL → Cypher** is a no-op scaffold. T-Box OWL 2 RL still runs (RDFLib upstream). Full translator = follow-up PR.
- **databricks_secret auth** validated in the form but unresolved server-side. Follow-up.
- **Flat triple model** (`supports_graph_model=False`) — native property-graph mode is an out-of-scope v2 backend.

Ask: review & merge into `develop`

12 commits on `feature/neo4j-graphdb-skeleton`. The five most recent (`cc679b9` → `0ba4ca6`) are the engine-aware label fixes. Reviewer to optionally re-run `tests/integration/neo4j_e2e_smoke.py` against any Neo4j endpoint.